

Pandas — LEVEL 6: Adding and Removing Data

Line-by-line code + output + examples + easy interview explanations

1. Add Column

Code

```
import pandas as pd

df = pd.DataFrame({
    "Name": ["Amit", "Riya", "John"],
    "Age": [22, 24, 23]
})

df["City"] = ["Pune", "Mumbai", "Delhi"]

print(df)
```

Output

	Name	Age	City
0	Amit	22	Pune
1	Riya	24	Mumbai
2	John	23	Delhi

Explanation: `df["City"]` creates a new column. Each value is placed in the matching row.

Easy interview explanation: Use `df['column'] = values` to add a column.

2. Add Multiple Columns

Code

```
df["Salary"] = [30000, 40000, 35000]
df["Department"] = ["IT", "HR", "Finance"]

print(df)
```

Output

	Name	Age	City	Salary	Department
0	Amit	22	Pune	30000	IT
1	Riya	24	Mumbai	40000	HR
2	John	23	Delhi	35000	Finance

Explanation: Two columns are added by making two assignments.

Easy interview explanation: Multiple columns can also be added with `assign()`.

3. Add Row

Code

```
new_row = pd.DataFrame({
    "Name": ["Neha"],
    "Age": [25],
    "City": ["Nashik"]
})

df = pd.concat([df, new_row], ignore_index=True)

print(df)
```

Output

	Name	Age	City
0	Amit	22	Pune
1	Riya	24	Mumbai
2	John	23	Delhi
3	Neha	25	Nashik

Explanation: `pd.concat()` combines DataFrames. `ignore_index=True` creates a new index.

Easy interview explanation: Use `pd.concat()` to add rows in modern pandas.

4. assign()

Code

```
df = pd.DataFrame({
    "Name": ["Amit", "Riya", "John"],
    "Age": [22, 24, 23]
})

df = df.assign(
    City=["Pune", "Mumbai", "Delhi"],
    Salary=[30000, 40000, 35000]
)

print(df)
```

Output

	Name	Age	City	Salary
0	Amit	22	Pune	30000
1	Riya	24	Mumbai	40000
2	John	23	Delhi	35000

Explanation: `assign()` adds one or more columns and returns a new DataFrame.

Easy interview explanation: `assign()` is useful for adding several columns and method chaining.

5. insert()

Code

```
df = pd.DataFrame({
    "Name": ["Amit", "Riya", "John"],
    "Age": [22, 24, 23],
    "City": ["Pune", "Mumbai", "Delhi"]
})

df.insert(1, "Salary", [30000, 40000, 35000])

print(df)
```

Output

	Name	Salary	Age	City
0	Amit	30000	22	Pune
1	Riya	40000	24	Mumbai
2	John	35000	23	Delhi

Explanation: `insert(1, ...)` puts Salary at position 1, so it becomes the second column.

Easy interview explanation: `insert()` is used when the column position matters.

6. Remove Column using drop()

Code

```
df = pd.DataFrame({
    "Name": ["Amit", "Riya", "John"],
    "Age": [22, 24, 23],
    "City": ["Pune", "Mumbai", "Delhi"]
})

df = df.drop("Age", axis=1)

print(df)
```

Output

```
   Name  City
0  Amit  Pune
1  Riya Mumbai
2  John  Delhi
```

Explanation: axis=1 means columns, so Age is removed.

Easy interview explanation: axis=1 → columns; axis=0 → rows.

7. Remove Multiple Columns

Code

```
df = pd.DataFrame({
    "Name": ["Amit", "Riya", "John"],
    "Age": [22, 24, 23],
    "City": ["Pune", "Mumbai", "Delhi"]
})

df = df.drop(["Age", "City"], axis=1)

print(df)
```

Output

```
   Name
0  Amit
1  Riya
2  John
```

Explanation: Pass a list of column names to remove multiple columns.

Easy interview explanation: Use drop(['col1', 'col2'], axis=1).

8. Remove Row using drop()

Code

```
df = pd.DataFrame({
    "Name": ["Amit", "Riya", "John"],
    "Age": [22, 24, 23]
})

df = df.drop(1)

print(df)
```

Output

```
   Name  Age
0  Amit   22
2  John   23
```

Explanation: drop(1) removes the row with index label 1. The index is not automatically reset.

Easy interview explanation: drop() can remove a row using its index label.

9. Remove Multiple Rows

Code

```
df = pd.DataFrame({
    "Name": ["Amit", "Riya", "John"],
    "Age": [22, 24, 23]
})

df = df.drop([0, 2])

print(df)
```

Output

```
   Name  Age
1  Riya   24
```

Explanation: Passing a list of index labels removes multiple rows.

Easy interview explanation: Use `drop([0, 2])` for multiple rows.

10. drop_duplicates()

Code

```
df = pd.DataFrame({
    "Name": ["Amit", "Riya", "Amit", "John"],
    "Age": [22, 24, 22, 23]
})

df = df.drop_duplicates()

print(df)
```

Output

```
   Name  Age
0  Amit   22
1  Riya   24
3  John   23
```

Explanation: `drop_duplicates()` removes duplicate rows and keeps the first occurrence by default.

Easy interview explanation: It is commonly used to remove repeated records.

11. Duplicates using subset

Code

```
df = pd.DataFrame({
    "Name": ["Amit", "Riya", "Amit", "John"],
    "Age": [22, 24, 25, 23]
})

df = df.drop_duplicates(subset=["Name"])

print(df)
```

Output

```
   Name  Age
0  Amit   22
1  Riya   24
3  John   23
```

Explanation: subset=['Name'] checks duplicates using only Name.

Easy interview explanation: subset defines which columns should be checked for duplicates.

12. Rename Columns using rename()

Code

```
df = pd.DataFrame({
    "Name": ["Amit", "Riya"],
    "Age": [22, 24]
})

df = df.rename(columns={
    "Name": "Student_Name",
    "Age": "Student_Age"
})

print(df)
```

Output

```
   Student_Name  Student_Age
0         Amit             22
1         Riya             24
```

Explanation: rename(columns={old:new}) changes selected column labels.

Easy interview explanation: rename() changes labels without changing the data.

13. Rename Index using rename()

Code

```
df = pd.DataFrame({
    "Name": ["Amit", "Riya"],
    "Age": [22, 24]
})

df = df.rename(index={
    0: "Student_1",
    1: "Student_2"
})

print(df)
```

Output

```
      Name  Age
Student_1  Amit  22
Student_2  Riya  24
```

Explanation: rename(index={old:new}) changes index labels.

Easy interview explanation: rename() can change both column and index labels.

14. Rename All Columns

Code

```
df = pd.DataFrame({
    "Name": ["Amit", "Riya"],
    "Age": [22, 24]
})

df.columns = ["Student_Name", "Student_Age"]
```

```
print(df)
```

Output

```
   Student_Name  Student_Age
0         Amit           22
1         Riya           24
```

Explanation: `df.columns` replaces all column names. The number of names must match the number of columns.

Easy interview explanation: Use `df.columns` when replacing every column name.

15. Interview Quick Revision

Code

```
# Add column
df["City"] = ["Pune", "Mumbai"]

# Add multiple columns
df = df.assign(Salary=[30000, 40000], Dept=["IT", "HR"])

# Add column at a position
df.insert(1, "Age", [22, 24])

# Remove column
df = df.drop("Age", axis=1)

# Remove row
df = df.drop(0)

# Remove duplicates
df = df.drop_duplicates()

# Rename columns
df = df.rename(columns={"City": "Location"})

# Rename index
df = df.rename(index={0: "A", 1: "B"})
```

Output

```
Remember:
axis=0 → rows
axis=1 → columns
assign() → add columns
insert() → add column at a position
drop() → remove rows or columns
drop_duplicates() → remove duplicate rows
rename() → change labels
```

Explanation: These are the main Level 6 operations you should know for interviews.

Easy interview explanation: Use these commands as your Level 6 interview revision.

LEVEL 6 COMPLETE

You have covered adding columns, multiple columns, rows, `assign()`, `insert()`, removing columns and rows, `drop_duplicates()`, and renaming columns and indexes.